

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

OPERATING SYSTEMS LABORATORY

Course Code: CSE309R01

Semester: V

Lab Manual

2026

ROHITH KANNA GV

128003214

A-44

SCHOOL OF COMPUTING
SHANMUGHA ARTS, SCIENCE, TECHNOLOGY AND RESEARCH ACADEMY
(SASTRA Deemed to be University)
Tirumalaisamudram, Thanjavur-613 401

Exercise 13 Program to simulate page replacement algorithms and to compute number of page faults

Objectives:

To simulate page replacement algorithms.

Prerequisite/ Tools Required:

Knowledge of Paging concepts, replacement algorithms

Theory or Concept:

Page replacement algorithms are an important part of virtual memory management and it helps the OS to decide which memory page can be moved out making space for the currently needed page. However, the ultimate objective of all page replacement algorithms is to reduce the number of page faults.

FIFO-This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

LRU-In this algorithm page will be replaced which is least recently used

OPTIMAL- In this algorithm, pages are replaced which would not be used for the longest duration of time in the future. This algorithm will give us less page fault when compared to other page replacement algorithms.

Procedure /Algorithm:

Step 1: Input the number of pages and number of frames

Step 2: Input list of page numbers into an array a[i]

Step 3: Implement the page replacement algorithm

FIFO/LRU/Optimal Step 4: Calculate the number of page faults for the given page numbers Step 5: Print the results.

CODE:

fifo.c

```
#include <stdio.h>
#include <stdbool.h>

bool isInMemory(int page, int frames[], int n) {
    for (int i = 0; i < n; i++)
        if (frames[i] == page)
            return true;
    return false;
}

int simulateFIFO(int pages[], int n, int f) {
    int faults = 0, frames[f], ptr = 0;
```

```
    for (int i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nIncoming\t");
    for (int i = 0; i < f; i++)
        printf("Frame %d\t", i + 1);
    printf("\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t\t", pages[i]);

        if (!isInMemory(pages[i], frames, f)) {
            faults++;
            frames[ptr] = pages[i];
            ptr = (ptr + 1) % f;
        }

        for (int j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("-\t\t");
            else
                printf("%d\t\t", frames[j]);
        }
        printf("\n");
    }

    return faults;
}

int main() {
    int n, f;

    printf("Enter number of memory frames: ");
    scanf("%d", &f);
    printf("Enter number of page references: ");
    scanf("%d", &n);

    if (f <= 0 || n <= 0)
        return 1;

    int pages[n];

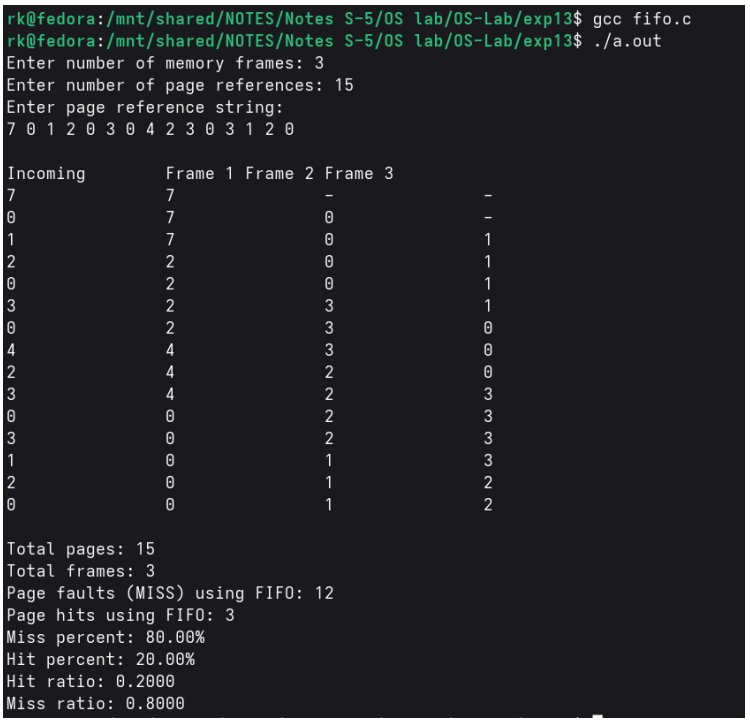
    printf("Enter page reference string:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    int faults = simulateFIFO(pages, n, f);
```

```
    int hits = n - faults;

    printf("\nTotal pages: %d\nTotal frames: %d\n", n, f);
    printf("Page faults (MISS) using FIFO: %d\n", faults);
    printf("Page hits using FIFO: %d\n", hits);
    printf("Miss percent: %.2f%%\n", (float)faults / n * 100);
    printf("Hit percent: %.2f%%\n", (float)hits / n * 100);
    printf("Hit ratio: %.4f\n", (float)hits / n);
    printf("Miss ratio: %.4f\n", (float)faults / n);

    return 0;
}
```



```
rk@fedora:/mnt/shared/NOTES/Notes S-5/OS lab/OS-Lab/exp13$ gcc fifo.c
rk@fedora:/mnt/shared/NOTES/Notes S-5/OS lab/OS-Lab/exp13$ ./a.out
Enter number of memory frames: 3
Enter number of page references: 15
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 1 2 0

Incoming        Frame 1 Frame 2 Frame 3
7               7               -               -
0               7               0               -
1               7               0               1
2               2               0               1
0               2               0               1
3               2               3               1
0               2               3               0
4               4               3               0
2               4               2               0
3               4               2               3
0               0               2               3
3               0               2               3
1               0               1               3
2               0               1               2
0               0               1               2

Total pages: 15
Total frames: 3
Page faults (MISS) using FIFO: 12
Page hits using FIFO: 3
Miss percent: 80.00%
Hit percent: 20.00%
Hit ratio: 0.2000
Miss ratio: 0.8000
```

lru.c

```
#include <stdio.h>
#include <stdbool.h>

bool isInMemory(int page, int frames[], int n) {
    for (int i = 0; i < n; i++)
        if (frames[i] == page)
            return true;
    return false;
}

int simulateLRU(int pages[], int n, int f) {
    int faults = 0, frames[f], last_used[f];

    for (int i = 0; i < f; i++) {
        frames[i] = -1;
```

```
    last_used[i] = -1;
}

printf("\nIncoming\t");
for (int i = 0; i < f; i++)
    printf("Frame %d\t", i + 1);
printf("\n");

for (int i = 0; i < n; i++) {
    printf("%d\t\t", pages[i]);

    int j;
    if (isInMemory(pages[i], frames, f)) {
        for (j = 0; j < f; j++)
            if (frames[j] == pages[i]) {
                last_used[j] = i;
                break;
            }
    } else {
        faults++;
        int k = -1;

        for (j = 0; j < f; j++)
            if (frames[j] == -1) {
                k = j;
                break;
            }

        if (k == -1) {
            k = 0;
            for (j = 1; j < f; j++)
                if (last_used[j] < last_used[k])
                    k = j;
        }

        frames[k] = pages[i];
        last_used[k] = i;
    }

    for (j = 0; j < f; j++) {
        if (frames[j] == -1)
            printf("-\t\t");
        else
            printf("%d\t\t", frames[j]);
    }
    printf("\n");
}
```

```
    return faults;
}

int main() {
    int n, f;

    printf("Enter number of memory frames: ");
    scanf("%d", &f);
    printf("Enter number of page references: ");
    scanf("%d", &n);

    if (f <= 0 || n <= 0)
        return 1;

    int pages[n];

    printf("Enter page reference string:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    int faults = simulateLRU(pages, n, f);
    int hits = n - faults;

    printf("\nTotal pages: %d\nTotal frames: %d\n", n, f);
    printf("Page faults (MISS) using LRU: %d\n", faults);
    printf("Page hits using LRU: %d\n", hits);
    printf("Miss percent: %.2f%%\n", (float)faults / n * 100);
    printf("Hit percent: %.2f%%\n", (float)hits / n * 100);
    printf("Hit ratio: %.4f\n", (float)hits / n);
    printf("Miss ratio: %.4f\n", (float)faults / n);

    return 0;
}
```

```
rk@fedora:/mnt/shared/NOTES/Notes S-5/OS lab/OS-Lab/exp13$ gedit lru.c
rk@fedora:/mnt/shared/NOTES/Notes S-5/OS lab/OS-Lab/exp13$ gcc lru.c
rk@fedora:/mnt/shared/NOTES/Notes S-5/OS lab/OS-Lab/exp13$ ./a.out
Enter number of memory frames: 3
Enter number of page references: 15
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 1 2 0

Incoming        Frame 1 Frame 2 Frame 3
7               7               -               -
0               7               0               -
1               7               0               1
2               2               0               1
0               2               0               1
3               2               0               3
0               2               0               3
4               4               0               3
2               4               0               2
3               4               3               2
0               0               3               2
3               0               3               2
1               0               3               1
2               2               3               1
0               2               0               1

Total pages: 15
Total frames: 3
Page faults (MISS) using LRU: 12
Page hits using LRU: 3
Miss percent: 80.00%
Hit percent: 20.00%
Hit ratio: 0.2000
Miss ratio: 0.8000
```

opt.c

```
#include <stdio.h>
#include <stdbool.h>

bool isInMemory(int page, int frames[], int n) {
    for (int i = 0; i < n; i++)
        if (frames[i] == page)
            return true;
    return false;
}

int findOptimal(int pages[], int n, int frames[], int f, int start) {
    int index = -1, farthest = -1;

    for (int i = 0; i < f; i++) {
        int j;

        for (j = start; j < n; j++)
            if (frames[i] == pages[j])
                break;

        if (j == n)
            return i;
```

```
        if (j > farthest) {
            farthest = j;
            index = i;
        }
    }

    return index;
}

int simulateOptimal(int pages[], int n, int f) {
    int faults = 0, frames[f];

    for (int i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nIncoming\t");
    for (int i = 0; i < f; i++)
        printf("Frame %d\t", i + 1);
    printf("\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t\t", pages[i]);

        if (!isInMemory(pages[i], frames, f)) {
            faults++;
            int k = -1;

            for (int j = 0; j < f; j++)
                if (frames[j] == -1) {
                    k = j;
                    break;
                }

            if (k == -1)
                k = findOptimal(pages, n, frames, f, i + 1);

            frames[k] = pages[i];
        }

        for (int j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("-\t\t");
            else
                printf("%d\t\t", frames[j]);
        }
        printf("\n");
```

```
    }

    return faults;
}

int main() {
    int n, f;

    printf("Enter number of memory frames: ");
    scanf("%d", &f);
    printf("Enter number of page references: ");
    scanf("%d", &n);

    if (f <= 0 || n <= 0)
        return 1;

    int pages[n];

    printf("Enter page reference string:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    int faults = simulateOptimal(pages, n, f);
    int hits = n - faults;

    printf("\nTotal pages: %d\nTotal frames: %d\n", n, f);
    printf("Page faults (MISS) using Optimal: %d\n", faults);
    printf("Page hits using Optimal: %d\n", hits);
    printf("Miss percent: %.2f%%\n", (float)faults / n * 100);
    printf("Hit percent: %.2f%%\n", (float)hits / n * 100);
    printf("Hit ratio: %.4f\n", (float)hits / n);
    printf("Miss ratio: %.4f\n", (float)faults / n);

    return 0;
}
```

```
rk@fedora:/mnt/shared/NOTES/Notes S-5/OS lab/OS-Lab/exp13$ gedit opt.c
rk@fedora:/mnt/shared/NOTES/Notes S-5/OS lab/OS-Lab/exp13$ gcc opt.c
rk@fedora:/mnt/shared/NOTES/Notes S-5/OS lab/OS-Lab/exp13$ ./a.out
Enter number of memory frames: 3
Enter number of page references: 15
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 1 2 0

Incoming        Frame 1 Frame 2 Frame 3
7               7               -               -
0               7               0               -
1               7               0               1
2               2               0               1
0               2               0               1
3               2               0               3
0               2               0               3
4               2               4               3
2               2               4               3
3               2               4               3
0               2               0               3
3               2               0               3
1               2               0               1
2               2               0               1
0               2               0               1

Total pages: 15
Total frames: 3
Page faults (MISS) using Optimal: 8
Page hits using Optimal: 7
Miss percent: 53.33%
Hit percent: 46.67%
Hit ratio: 0.4667
Miss ratio: 0.5333
```

Algorithm	Faults	Hits	Hit %	Miss %
FIFO	12	3	20.00%	80.00%
LRU	12	3	20.00%	80.00%
Optimal	8	7	46.67%	53.33%

RESULT:

The FIFO, LRU and Optimal page replacement algorithms were implemented and executed successfully. The page faults, page hits, hit ratio and miss ratio were calculated and compared for the given page reference string.